assigned. In Ruby, a line of code like `2 + "3"` will be an error, but the error will happen at run time rather than compile time. In most dynamic languages the code `x + y` will have different behavior if `x` and `y` are numbers than if they are strings, and this behavior will be determined based on the value of `x` each time the line of code is executed.

TypeScript turns JavaScript into a statically typed language. Whether or not this is a good change is a surprisingly hard question to answer empirically. Both the general case of static versus dynamic languages and the specific case of TypeScript versus JavaScript are debated endlessly, with actual data difficult to come by. Creating any kind of valid, reproducible, scientific evidence about the general usefulness of programming languages is challenging.

There are a few points that are not disputed…much:

- A static language will catch errors in compilation that would otherwise potentially remain uncaught.
- Static languages generally are more verbose than dynamic languages, and there is sometimes a time cost to getting the compiler to agree that what you want to do is valid. More modern static languages use type inference to minimize the extra verbosity.
- Dynamic languages are generally more flexible and are usually considered easier to write code in, at least for small programs.
- Static languages provide more information to code analysis tools, so editor and tool support is easier to create. They also provide more meta-information in general, which can be valuable as communication about the code on larger teams.

The idea is that in a good static typing system, the benefits of tool support, communication, and error prevention will outweigh the costs of yelling at the compiler trying to get it to let you do what you want. In the general case of static versus dynamic languages, I think there's a lot of room for debate. In the specific case of TypeScript versus JavaScript, I think there is good reason to think there's